

Computational Linguistics Team Laboratory

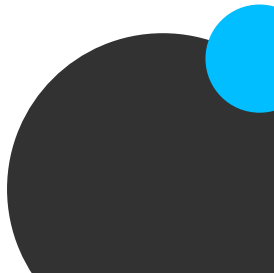
Week 2: Baseline Method and Implementation Hints

Dr. Agnieszka Faleńska

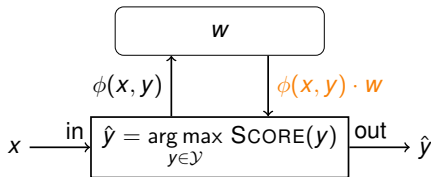
Institute for Natural Language Processing



Universität Stuttgart

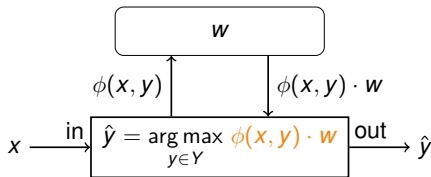


The Big Picture and Notation – Classification Tasks



- ▶ $x \in \mathcal{X}$ – an input text
- ▶ $\hat{y} \in \mathcal{Y}$ – a predicted class (label) for x
- ▶ $\phi : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}^n$ – feature extraction function
- ▶ $w \in \mathbb{R}^n$ – vector with weights

The Big Picture and Notation – Classification Tasks



- ▶ $x \in \mathcal{X}$ – an input text
- ▶ $\hat{y} \in \mathcal{Y}$ – a predicted class (label) for x
- ▶ $\phi : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}^n$ – feature extraction function
- ▶ $w \in \mathbb{R}^n$ – vector with weights

Find the class y that maximizes the scalar product between the feature extraction function ϕ and the weight vector w

Adaptation for Sequence Labelling Tasks

Input:	I	love	programming	and	Python
Output:	PRP	VBP	NN	CC	NNP

- ▶ Convert the data into k-grams with POS labels:
 - ▶ "– I love" $\rightarrow$ PRP
 - ▶ "I love programming" $\rightarrow$ VBP
 - ▶ "love programming and" $\rightarrow$ NN
 - ▶ ...
- ▶ $x \in \mathcal{X}$ – an input k-gram
- ▶ $\hat{y} \in \mathcal{Y}$ – a predicted class (label) for x
- ▶ k is a hyperparameter

Adaptation for Sequence Labelling Tasks

Input:	I	love	programming	and	Python
Output:	PRP	VBP	NN	CC	NNP

- ▶ Convert the data into k-grams with POS labels:
 - ▶ "– I love" $\rightarrow$ PRP
 - ▶ "I love programming" $\rightarrow$ VBP
 - ▶ "love programming and" $\rightarrow$ NN
 - ▶ ...
- ▶ $x \in \mathcal{X}$ – an input k-gram
- ▶ $\hat{y} \in \mathcal{Y}$ – a predicted class (label) for x
- ▶ k is a hyperparameter

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Feature Function ϕ

- ▶ We have a pair $\langle x, y \rangle$, where:
 - ▶ $x \in \mathcal{X}$ - an input text
 - ▶ $y \in \mathcal{Y}$ - a class
- ▶ ϕ maps pairs $\langle x, y \rangle$ to a *high-dimensional feature vector*

$$\phi : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}^n$$

- ▶ ϕ is composed from individual feature functions:

$$\phi(x, y) = [\phi_1(x, y), \phi_2(x, y), \dots, \phi_n(x, y)]$$

where $\phi_i : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$

How does ϕ look like?

	x_1	x_2	x_3	x_4	x_5	x_6	...
ϕ_1	1	1	0	0	0	1	
ϕ_2	1	1	0	1	0	0	
ϕ_3	1	1	0	1	1	1	
ϕ_4	0	1	0	0	0	0	
ϕ_5	1	0	0	0	0	0	
ϕ_6	1	0	1	1	1	1	
ϕ_7	1	0	1	1	1	0	
...							

Each x is represented as a **binary vector** $\in \{0, 1\}^n$.

How to Design a Feature Function?

- ▶ Instead of selecting every individual ϕ_i we decide on the type of information they will use
- ▶ Possible values for these features come from training data
- ▶ Most obvious type of features? words

Feature type: words

	x_1	x_2	x_3	x_4	x_5	x_6	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

How to Design a Feature Function?

- ▶ Instead of selecting every individual ϕ_i we decide on the type of information they will use
- ▶ Possible values for these features come from training data
- ▶ Most obvious type of features? words (unigrams)
- ▶ Other ideas?
bigrams, trigrams, lemmas, character n-grams, word stems, POS tags, POS tag n-grams, dependency parse relations, presence of positive/negative words, emoticons, word clusters, topic categories, . . .

Multiple feature types

	x_1	x_2	x_3	x_4	x_5	x_6	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
...							
sad	1	0	1	1	1	0	
happy	1	0	1	1	1	0	
...							
NOUN	1	0	1	1	1	0	
VERB	1	0	1	1	1	0	
...							

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

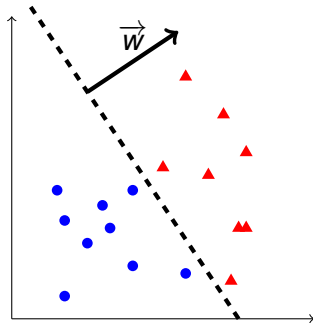
Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

What is a Linear Classifier (2-D Case)?



- ▶ A hyperplane that separates objects into two classes
- ▶ A data set is linearly separable iff there is such a hyperplane
- ▶ The hyperplane can be described by a vector $\vec{w} \in \mathbb{R}^n$ (it is the vector that stands orthogonal on the plane)
- ▶ $\vec{w}$ is called the **weight vector**

The Perceptron Algorithm

?

There is many machine learning algorithms to find $\vec{w}$

- ▶ SVM
- ▶ neural networks
- ▶ ...

This class: the perceptron algorithm from ?

- ▶ supervised learning algorithm
- ▶ online learning algorithm
- ▶ original version is for binary classification

Binary Classification with Perceptron

Assume we have:

- ▶ $D = \{\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle, \dots, \langle x_N, y_N \rangle\}$ – training set
 - ▶ $x_i \in \mathcal{X}$ – input objects
 - ▶ $y_i \in \mathcal{Y}$ – class labels
 - ▶ $\mathcal{Y} = \{-1, 1\}$ – binary classification (● = -1, ▲ = 1)
- ▶ $\phi : \mathcal{X} \rightarrow \{0, 1\}^n$ – feature extraction function
 - ▶ takes an input object $x \in \mathcal{X}$ and returns a feature vector in $\{0, 1\}^n$

Standard operations:

- ▶ Dot-product (scalar product): $\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i \cdot b_i$
- ▶
$$\text{SIGN}(c) = \begin{cases} 1 & \text{if } c \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Binary Classification with Perceptron

Pseudocode and Example

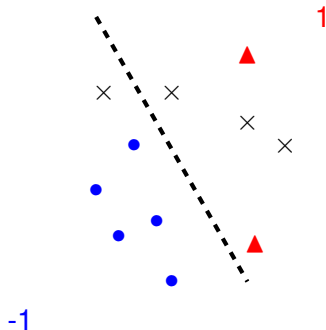
Require: $D = \{\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle, \dots, \langle x_N, y_N \rangle\}$

Require: Number of iterations T

Ensure: Weight vector w

```
1:  $w = \vec{0}$ 
2: for  $t \in 1..T$  do
3:   for  $i \in 1..N$  do
4:      $\hat{y}_i = \text{SIGN}(\phi(x_i) \cdot w)$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $w = w + y_i \cdot \phi(x_i)$  # Update
7:     end if
8:   end for
9: end for return  $w$ 
```

Example:



Perceptron Update

Zoom in

```
5:  $\hat{y}_i = \text{SIGN}(\phi(x_i) \cdot w)$  # Predict  
6: if  $\hat{y}_i \neq y_i$  then # Correct?  
7:    $w = w + y_i \cdot \phi(x_i)$  # Update  
8: end if
```

$$\text{Decision boundary: } \text{SIGN}(c) = \begin{cases} 1 & \text{if } c \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

► Prediction: -1 correct: 1

► The scalar product was lower than 0 $\Rightarrow$ increase weights
 $w = w + 1 \cdot \phi(x_i)$

► Prediction: 1 correct: -1

► The scalar product was higher than 0 $\Rightarrow$ decrease weights
 $w = w - 1 \cdot \phi(x_i)$

Perceptron Update

Example

5: $\hat{y}_i = \text{SIGN}(\phi(x_i) \cdot w)$ # Predict
6: **if** $\hat{y}_i \neq y_i$ **then** # Correct?
7: $w = w + y_i \cdot \phi(x_i)$ # Update
8: **end if**

Current state:

► $w = [1, -1, 0, -3]$

Next instance:

► $\phi(x_i) = [1, 0, 0, 1]$

► $y_i = 1$

► Predict:

► $\phi(x_i) \cdot w = 1 * 1 + 0 * -1 + 0 * 0 + 1 * -3 = -2$

► $\text{SIGN}(-2) = -1$

► Correct?

► $-1 \neq 1$ **No!**

► Update:

► $w = w + 1 \phi(x_i) = [1, -1, 0, -3] + 1[1, 0, 0, 1] = [2, -1, 0, -2]$

Perceptron Update

Example – an intuition

The instance:

- ▶ $\phi(x_i) = [1, 0, 0, 1]$
- ▶ $y_i = 1$

Was:

- ▶ $w = [1, -1, 0, -3]$

Is:

- ▶ $w = [2, -1, 0, -2]$

Weights increased for features present in $\phi(x_i)$ because they are important in predicting $y = 1$

Extending Perceptron to Multi-class Classification

Require: $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$

Require: Number of iterations T

Ensure: Weight vectors w_y for every $y \in Y$

```
1:  $w_y = \vec{0}$  for every  $y \in Y$ 
2: for  $t \in 1..T$  do
3:   for  $i \in 1..N$  do
4:      $\hat{y}_i = \arg \max_y (w_y \cdot \phi(x_i))$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $w_{y_i} = w_{y_i} + \phi(x_i)$ 
7:        $w_{\hat{y}_i} = w_{\hat{y}_i} - \phi(x_i)$ 
8:     end if
9:   end for
10: end for return  $w_y$  for every  $y \in Y$ 
```

► Implementation note - W is in fact a matrix of class vectors.

Extending Perceptron to Multi-class Classification

Require: $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$

Require: Number of iterations T

Ensure: Weight matrix W

```
1:  $W[y] = \vec{0}$  for every  $y \in Y$ 
2: for  $t \in 1..T$  do
3:   for  $i \in 1..N$  do
4:      $\hat{y}_i = \arg \max_y (W[y] \cdot \phi(x_i))$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $W[y_i] = W[y_i] + \phi(x_i)$ 
7:        $W[\hat{y}_i] = W[\hat{y}_i] - \phi(x_i)$ 
8:     end if
9:   end for
10: end for return  $W$ 
```

► Implementation note - W is in fact a matrix of class vectors.

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

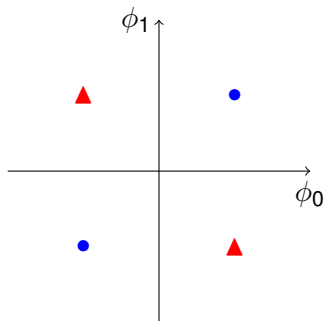
Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Properties of Perceptron (I)

- ▶ This is a **linear model**, we *sum* over all features

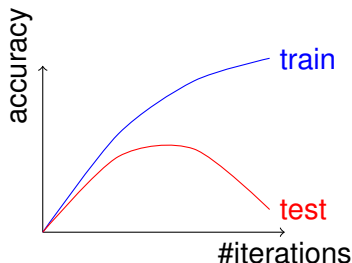


- ▶ Use combination of features to overcome that

$$\phi_0 > 0 \text{ and } \phi_1 < 0$$

Properties of Perceptron (II)

- ▶ Expressive feature combinations might cause model overfit
- ▶ Train and test accuracy changes differently:
 - ▶ train accuracy will always increase
 - ▶ test accuracy will drop at some point



- ▶ Need to find an appropriate number of iterations
- ▶ Or monitor the accuracy on dev set and early stop

Properties of Perceptron (III)

- ▶ Online algorithm - the result might change with the order of training data
- ▶ The later data points count more than the earlier points
 - ▶ very bad case - all positive examples before negative ones
 - shuffle training points before training
 - shuffle training points every iteration - even better
- ▶ We need better generalization
 - Averaged Perceptron

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Averaged Perceptron

in ??

We need better generalization:

- ▶ average all weight vectors seen during training
- ▶ return the averaged vector

```
1:  $w = \vec{0}$ 
2: for  $t \in 1..T$  do
3:   for  $i \in 1..N$  do
4:      $\hat{y}_i = \text{SIGN}(w \cdot \phi(x_i))$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $w = w + y_i \cdot \phi(x_i)$ 
7:     end if
8:   end for
9: end for
10: return  $w$ 
```

```
1:  $w = \vec{0}$ 
2: for  $t \in 1..T$  do
3:   for  $i \in 1..N$  do
4:      $\hat{y}_i = \text{SIGN}(w \cdot \phi(x_i))$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $w = w + y_i \cdot \phi(x_i)$ 
7:     end if
8:   end for
9: end for
10: return  $w_{\text{average}}$ 
```

How to build $w_{average}$

```
1:  $w = \vec{0}$ 
2:  $w_{sum} = \vec{0}$ 
3: for  $t \in 1..T$  do
4:   for  $i \in 1..N$  do
5:      $w_{sum} = w_{sum} + w$ 
6:      $\hat{y}_i = \text{SIGN}(w \cdot \phi(x_i))$ 
7:     if  $\hat{y}_i \neq y_i$  then
8:        $w = w + y_i \cdot \phi(x_i)$ 
9:     end if
10:   end for
11: end for
12:  $w_{average} = \frac{1}{N \cdot T} \cdot w_{sum}$ 
13: return  $w_{average}$ 
```

Makes an update for every data point, even if the prediction $\hat{y}_i$ was correct and w did not change

How to build $w_{average}$

```
1:  $w = \vec{0}$ 
2:  $u = \vec{0}$  # cached weights
3:  $q = 0$  # example counter
4: for  $t \in 1..T$  do
5:   for  $i \in 1..N$  do
6:      $q = q + 1$  # increment regardless of update
7:      $\hat{y}_i = \text{SIGN}(w \cdot \phi(x_i))$ 
8:     if  $\hat{y}_i \neq y_i$  then
9:        $\Delta = y_i \cdot \phi(x_i)$ 
10:       $w = w + \Delta$  # re-use  $\Delta$ 
11:       $u = u + q \cdot \Delta$  # update cached weights
12:    end if
13:  end for
14: end for
15:  $w_{average} = w - \frac{1}{q} \cdot u$ 
16: return  $w_{average}$ 
```

Updates only when the weight vector changes.

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Big picture

Train

1. Read training data
2. Extract features
3. Perceptron training
4. Save model to file

Test (predict)

1. Read model from file
2. For every text in test data
 - 2.1 Extract features and apply model
 - 2.2 Write result to output file

► Common functionality

- Reading input sentences
- Extracting features
- Make a prediction given weight vector

READER
EXTRACTFEATURES

General workflow

- ▶ Below we will summarize a general workflow how to develop a classifier
 - ▶ some content needs adaptation to sequence labelling tasks
- ▶ The workflow is a guide how to go about and make **sanity checks** along the way
- ▶ **It's just one way to implement it, not the only way**

General workflow

1. Input/Output and text objects
2. Evaluation script
3. Basic feature extraction
4. Code for training and saving model
5. Code for reading saved model and testing

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

1. Input/Output and sentence objects

- ▶ **Text:** Object to represent an input text
- ▶ Depends on what you have in your data and your task

```
class Token:
    form: str = ...
    pos: str = ...
    head: int = ...
class Text:
    tokens: List[Token] = ...
```

1. Input/Output and text objects

- ▶ **Reader:** Reads a file and returns text objects

```
def read(filename: str): -> List[Text]
  ...
```

- ▶ **Writer:** Writes a text object to a file

```
def write(filename: str, texts: List[Text]):
  ...
```

1. Input/Output and sentence objects

- ▶ Sanity-check???
 - ▶ Read a file and write it back out to a new file.
 - ▶ Make sure the two files are identical (e.g., by running `diff`)
- ▶ Attention
 - ▶ Don't assume there are always gold-standard labels
 - ▶ Test files might have only texts (no labels)

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
- 2. Evaluation script**
3. Basic feature extraction
4. Training
5. Testing

Future steps

2. Evaluation script

- ▶ Script: reads two files and calculates the evaluation metric:

```
evaluate.py correct.txt predicted.txt
```

- ▶ Somewhere internally:

```
def F1(correct: List[Text], predicted: List[Text]): ->  
float  
    ...
```

2. Evaluation script

Sanity-checks???

```
evaluate.py correct.txt correct.txt
```

```
evaluate.py correct.txt random_labels.txt
```

```
evaluate.py dev.txt dev_pred.txt
```

- ▶ you need to first prepare files: random_labels.txt
- ▶ some datasets come with dev_pred.txt

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences

2. Evaluation script

3. Basic feature extraction

4. Training

5. Testing

Future steps

3. Basic feature extraction

		x_1	x_2	x_3	x_4	x_5	x_6	...
word:	Anthony	1	1	0	0	0	1	
word:	Brutus	1	1	0	1	0	0	
word:	sad	1	1	0	1	1	1	
word:	Calpurnia	0	1	0	0	0	0	
word:	Cleopatra	1	0	0	0	0	0	
	...							
emotion:	sad	1	0	1	1	1	0	
emotion:	happy	1	0	1	1	1	0	
	...							
POS:	NOUN	1	0	1	1	1	0	
POS:	VERB	1	0	1	1	1	0	
	...							

- ▶ Watch out for feature collisions!
- ▶ Add feature types to feature names

3. Basic feature extraction

Function for extracting features from text:

```
def features(text: Text): -> List[String]
```

► Example:

```
"I love programming" → [ "word:I", "word:love",  
"word:programming", "POS:VERB", "POS:NOUN", ...]
```

3. Basic feature extraction

- ▶ Need to map features into proper places in the vectors
- ▶ Implement this as a dictionary that maps strings to integers

```
class FeatureMapping:  
    map: Dict[str, int]  
    frozen: bool          //freeze the mapping after training
```

- ▶ Example:

```
"word:I" → 1  
"word:love" → 2  
"POS:NOUN" → 3  
...
```

- ▶ Populate the map while extracting features for training
- ▶ While testing, no new features can be added (freeze after training)

3. Basic feature extraction

- ▶ Sanity check???
- ▶ Iterate over all items of the map and check that
 1. Each index is used exactly once
 2. Each index from 0 to $d - 1$ is used
- ▶ Print key,value pairs to STDOUT and see that it looks reasonable

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
- 4. Training**
5. Testing

Future steps

4. Code for training

- ▶ Use the perceptron algorithm to train a weight vector
- ▶ Before training, read the training data and map it to *Instances*
 - ▶ An Instance corresponds to an (x, y) pair that the perceptron loops over
 - ▶ Instances don't need to deal with strings, but only with numbers
 - Reduces the memory footprint
 - Avoids repeating computations between iterations

4. Code for training

- ▶ What's in an instance?

```
class Instance:
    label: int                                //correct label
    feats: Dict[int, int]                     //sparse representation
```

- ▶ Need to map labels (classes) to integers: 0, 1, 2, ...
- ▶ Use a sparse representation for the feature vector:
 - ▶ Key is the dimension in the vector
 - ▶ Value is the value

Quiz: How to represent [0,0,0,0,1,0,0,0,2,0,0,0]

Ans: $\{4 \rightarrow 1, 8 \rightarrow 2\}$

4. Code for training

► Big picture

```
featmap = FeatureMapping()           //Establish feature mapping
featmap.extract(train_texts)         // Populate feature mapping
featmap.frozen = True                //Freeze feature mapping
weights: List[List[float]] = ...     //Create weight matrix
instances = get_instances(train_texts)

for i in range(epochs):
    for inst in instances:
        ...                           perceptron training
```

4. Code for training

- ▶ Sanity check???
 - ▶ Try training! (and expect it to break because of your bugs!)
 - ▶ While training, print the percentage of correctly classified instances in this iteration (every 500 instances)
- ▶ When training the first time,
 - ▶ Use a subset of the training data
 - ▶ Expect it to take some time to train!

```
for i in range(epochs)
    total = 0
    correct = 0
    for inst in instances:
        ...                               //perceptron training
        total += 1
        if(correct prediction):
            correct += 1
    if total%500==0:
        accuracy = 100.0 * correct / total
        print("total:", total, "accuracy:", accuracy)
```

4. Code for training (saving model)

► How to save a model

```
import cPickle, gzip

outfile = ...                                //Output file

stream = gzip.open(outfile,'wb')
cPickle.dump(featmap,stream,-1)
cPickle.dump(weights,stream,-1)
stream.close()
```

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
- 5. Testing**

Future steps

5. Code for testing (reading model)

► Reading in model

```
import cPickle, gzip

infile = ...                                //input file
stream = gzip.open(infile,'rb')
featmap = cPickle.load(stream)
weights = cPickle.load(stream)
stream.close()
```

5. Code for testing (prediction)

- ▶ Sanity check???
- ▶ Apply the model to the development set!
- ▶ Compute the evaluation metric
- ▶ Expect something lower than the dummy method (random behaviour)
- ▶ Debug
- ▶ Finally, the accuracy should be higher than the dummy method
 - ▶ Start improving the method (e.g., adding features) only when you are sure there are no bugs!

Table of Contents

Feature Extraction

Learning the Weight Vector

Perceptron Algorithm

Properties of Perceptron

Averaged Perceptron (extension)

Implementation Hints

1. IO and sentences
2. Evaluation script
3. Basic feature extraction
4. Training
5. Testing

Future steps

Experimentation

- ▶ Experiments are a way to prove your hypothesis if you cannot prove it formally
- ▶ A good and expressive experiment:
 - ▶ tests one hypothesis and only this one hypothesis
 - ▶ makes as few additional assumptions as possible
- ▶ Designing good experiments that show what they are supposed to is an art

Typical Hypotheses

- ▶ With this new method, we can predict labels more accurately
- ▶ With this new method, we can predict labels faster with equal accuracy
- ▶ With this feature, we can predict labels more accurately
- ▶ With this new method, we can make this new feature, which helps in predicting labels more accurately
- ▶ With this new learning method, we can learn models that predict labels more accurately
- ▶ Feature X is useful in language A,B,C for predicting better labels
- ▶ ...

Baselines

- ▶ A baseline is a model that does not use innovation X
- ▶ Models are compared to baselines to show that the difference between them is important
- ▶ A good experiment has strong baselines
- ▶ Often, the current state-of-the-art can/should be used as a baseline

Simple Example

- ▶ You implemented a sentiment analyser and a basic feature set
- ▶ You think that a feature that looks at the part-of-speech tag of the first word and its dependent might be useful
- ▶ You add the feature to the feature set
- ▶ You train the model on the training set and then test it on the **development set**
- ▶ You compare your evaluation metric on the baseline and check if they are higher

Quiz: What is the baseline?

Ans: Your tool without this feature

Conclusion

- ▶ Phase 2:
 - ▶ Implement evaluation script
 - ▶ Implement dummy method (our baseline for now)
 - ▶ Implement feature extraction and perceptron
 - ▶ Evaluate your method
- ▶ Phase 3:
 - ▶ Two baselines: dummy method and perceptron from Phase 2
 - ▶ Experiment with various extensions and ideas

References I

- Collins, M. (2002). Discriminative training methods for hidden markov models: Theory and experiments with perceptron algorithms. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing*, pages 1–8. Association for Computational Linguistics.
- Freund, Y. and Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning*, 37(3):277–296.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.